| Method/Input Size | **vectorMedian** | **listMedian** | **heapMedian** | **treeMedian** |
|---|---|---|---|---|
| **Input 1 (4000)** | 1,322 ms | 22,112 ms | 2,322 ms | 2,760 ms |
| **Input 2 (16000)** | 7,201 ms | 364,989 ms | 9,003 ms | 9,899 ms |
| **Input 3 (64000)** | 66,656 ms | 8,886,989 ms | 35.325 ms | 43,766 ms |

Algorithmic analysis of each algorithm, both in big-O terms and in more exact terms:

**listMedian Big-O Analysis:**

- Time Complexity**: O(n^2)**

**Explanation:**

- Each pop median operation in listMedian involves traversing the linked list from the beginning to find the median element, resulting in a linear search for each operation.
- As the number of elements increases, the time taken for each pop median operation increases quadratically due to the linear search, leading to an overall time complexity of O(n^2).
- The quadratic time complexity of O(n^2) indicates that the performance degradation becomes increasingly pronounced as the input size grows. This inefficiency can be problematic for large datasets, where the time taken for each pop median operation can become longer, limiting the scalability.

**vectorMedian Big-O Analysis:**

- Time Complexity**: O(n^2)**

**Explanation:**

- vectorMedian has an overall runtime complexity of O(n^2) where n is the size of our instruction count.
- Insertion in a vector using binary search to find the proper location is O(log n). However, since inserting in a vector requires shifting all elements after the insertion point, it becomes an O(n) operation.
- Similarly, for the pop operation, even though we can directly index the center (O(1)), removing an element requires shifting all subsequent elements back one space, making it an O(n) operation.
- Therefore, each operation (insert and pop) has a time complexity of O(n), and since we perform these operations for n inputs, the overall time complexity becomes O(n^2).

**heapMedian Big-O Analysis:**

- Time Complexity**: O(n log n)**

**Explanation:**

- Each insertion operation in the heapMedian function involves pushing elements into one of the two priority queues, which has a time complexity of O(log n).
- Additionally, during each pop median operation, the function requires popping elements from one of the priority queues and potentially rearranging elements in the other priority queue to maintain the heap property, also resulting in a time complexity of O(log n).
- Therefore, considering both insertions and pop median operations, the overall time complexity of heapMedian is O(n log n).

**treeMedian  Big-O Analysis:**

- Time Complexity**: O(n log n)**

**Explanation:**

- Each insertion operation in the treeMedian function involves traversing the AVL tree to find the appropriate position for insertion, which has a time complexity of O(log n).
- Additionally, each deletion operation in the treeMedian function involves finding the node to delete and rebalancing the AVL tree if necessary, also resulting in a time complexity of O(log n).
- Therefore, considering both insertions and deletions, and assuming a balanced AVL tree, the overall time complexity of treeMedian is O(n log n).

During the project, I noticed that the choice of data structure impacted memory usage. For example, while vectors and lists can require contiguous memory allocation, trees and heaps utilize more complex structures that may consume additional memory. This consideration is important, especially when dealing with large datasets or memory-constrained environments, as it can affect the overall performance and scalability of the program. The overall time complexity is O(n^2) due to the implementation of the vector and list methods. Both methods utilize insertion and deletion operations within their data structures, which inherently have linear complexities. As the number of elements (n) increases, the quadratic time complexity becomes apparent, leading to slower performance, especially with larger datasets. Despite variations in implementation details, the essential operations within these methods contribute to the O(n^2) time complexity observed across the project. The overall space complexity of this project is O(n) because each method employs data structures that require space proportional to the number of elements (n) in the input dataset. For instance, the vector, list, heap, and AVL tree methods all allocate memory dynamically to store the elements, resulting in linear space usage.